2322ENG: Engineering Programming

Programming Laboratory 4: Functions

Instructions:

This laboratory is designed to introduce you to functions. Functions are important for modularising your program to improve its readability, maintainability, and re-usability. They are also a crucial part of the software design process.

You should have read all "Functions" notes (first half of topic 3) and watched all of the associated videos prior to attending this lab!

Homework Question:

These questions should be completed *prior* to the lab session.

1. **Write a C function called *min* which takes 2 integers as input, and returns the smallest as the result.**
2. **Write a C function called *is_vowel* which takes a single character as an input and returns 1 if that character is a vowel ('a', 'e', 'i', 'o' or 'u') or 0 otherwise.**
3. **Write a C function called *is_lower* which takes a single character as input, and returns 1 if the character is a lower case letter ('a'-'z') or 0 otherwise.**
4. **Write a C *definition (prototype)* for a function called *power* which takes a float and an integer as inputs, and returns a float as a result.**
5. **Write the body of the function from (4). This function should raise the first argument to the power of the second, by multiplying it by itself that many times (don't use the inbuilt *pow* function, write your own). Do not worry about "special" cases like negative powers.**
6. **Write a C function called *display_dollar* which takes a floating point number as input, and displays this is a dollar value to the screen. This means that it should have a '$' in front of it, and be displayed to exactly two decimal places in non-scientific form. Your function should not put spaces or newlines before or after the output.**

Code re-usability and maintenance

The use of functions in programming provides for the modularisation of a large C program, in order to improve code re-usability and maintenance. Let us look at why these two features are important.

Suppose you have written a very efficient C program for computing the *square root* of a number and would like to re-use it in your future projects. Alternatively, your colleague would probably like to use it for their C project as well. Sending them your complete C program is tedious, since they would need to strip out the unnecessary code sections and copy the square root code into their program. Another problem is if we needed to compute lots of square roots in order to achieve the required result. This means we would need to copy and paste the code *multiple times,* which is also very tedious. Imagine what would happen if we found an error in our code. We would need to correct the error multiple times. Your colleague probably wouldn't be impressed if you asked them to make the correction multiple times either!

Both of these situations require a way of packaging C code, which performs a single important task,

into a self-contained module that can be *re-used* again later on. Also if an error is found, the correction is applied only to the module so subsequent programs that call this module will incorporate this fix as well. This allows the code to be easily maintained.

In C, the mechanism for packaging code into a self-contained module is called a *function*.

Functions in C

Up until now, all the C programs that you have written have contained one function called *main*. The main function is special because it is called automatically when your executable is run. It contains the code to be executed from the beginning to the end of the program. We *define* the main function by writing statements that are enclosed within the curly brackets {}.

We have also *called* numerous functions too. For example, *printf* is a function for writing output to the screen. The *scanf* function was used to read input from the keyboard. There were also the mathematical functions, such as *sin* and *cos*. These functions required *input* (in the form of a variable or constants) and some of them (such as sin and cos) returned an *output*. In general, C functions can accept *inputs* that are processed by statements within the function, and return the result as an *output*.

Notice the distinction between *defining* and *calling* a function. Defining a function allows us to specify what instructions are to be performed using C statements. Calling a function is when we wish to use it in our code.

Let us look at the general structure of a C function *definition*:

```
return_type function_name(input_parameters) /* This is the header */
{
      statement1;  /* body of the
      statement2;     function */
      …
      return (var);
}
```

Firstly, the top of the function is called the *function header*. The header contains important information about the function, such as its name, input parameters, etc. We need to give the function a name (function_name). This can be any name that has not been used for another function or variable. We specify the inputs to the function (input_parameters) as a comma-separated list of data types followed by variable names (much like how we declare variables, in fact).

The body of the function contains C statements (including variable declarations). Results are returned by using the *return* statement. The data type of the result to be returned by the function is specified by the return_type.

Here is an example of a function definition called add_numbers:

```
float add_numbers(float num1, float num2)
{
      float result;
```

```
        result = num1 + num2;

        return (result);
}
```

Notice in the body of a function that it contains a variable declaration. Just like in all the C code we have written previously, all variables that we use need to be declared with their data types. But notice that only the variable result needs to be declared. But what about num1 and num2?

- **Discuss with your classmates and demonstrator as to why num1 and num2 aren't declared as variables within the function.**

It is important to note that all variables declared within a function (e.g. num1, num2, result) are only visible to statements within that function. These are known as *local variables*. Similarly, all the variables that you declare in the main function are only visible or local to main. This is called the *scope* of the variables.

Where to put the function definition?

Now it is time to put the function definition inside a C program and call it from within the main function. There are two places to put the function definition: *before* main and *after* main. Both of these cases are shown below.

```
#include <stdio.h>

float add_numbers(float num1, float num2)
{
        float result;

        result = num1 + num2;

        return (result);
}

int main(void)
{
        float a = 3.5, b = 9.34, c;

        c = add_numbers(a, b);

        printf("The result is %f\n", c);

        return (0);
}
```

```
#include <stdio.h>

float add_numbers(float num1, float num2);

int main(void)
{
        float a = 3.5, b = 9.34, c;

        c = add_numbers(a, b);

        printf("The result is %f\n", c);

        return (0);
}

float add_numbers(float num1, float num2)
{
        float result;

        result = num1 + num2;
```

- **Type out both these programs and verify that they produce the same result.**

Notice that within the function call statement in main, we do not need to specify the datatypes of the inputs and output.

The guiding principle here is that C needs to know the name of the function, the types of the input arguments (if any), and the type of the returned output (if any), **before** calling the function.

In the first program, the function definition *appears before main*. This is acceptable because the header of the function definition provides the required information before it is called in main.

In the second program, the function definition *appears after main*. In order to satisfy the guiding principle above, we specify just the function header terminated by a semicolon (;). This is called a *function prototype (hightlighted in blue)*. You can think of it as 'declaring' the function before it is called.

- **Try removing the function prototype from the second program and see what compiler errors you get.**

The second program demonstrates the preferred way of using functions in C programming. That is, we place all function prototypes at the top before main, and the corresponding function definitions, either below main or in a separate file. The reason is that it facilitates code re-usability, where the function definition can be provided in a separate file or compiled object file (e.g. in an external library) and the function prototypes are stored in a *header file* (*.h files), which are included using #include. You've seen stdio.h and math.h before. These are header files that contain the function prototypes of all functions that are defined in their corresponding libraries. This allows one to hide or 'obfuscate' their code. For example, your colleague may only interested in calling your square root function, but not how you've implemented it. So you provide them with the header file (containing the function prototypes) as well as the object code, which they can link into their program.

Writing a factorial function

- **Now it's time to write your own function. Fill in the missing blanks to complete the factorial function.**

```
#include <stdio.h>

int find_factorial(int x);

int main(void)
{
     int x, fx;

     printf("Enter the value to find the factorial of: ");
     scanf("%i", &x);

     ????  /* call the factorial function and save result in fx */

     printf("The factorial of %i is %i\n", x, fx);
}
```

```
int find_factorial(int x)
{
     ???????
     ???????
     ….
     return (???);
}
```

Notice that for relatively small values of x, the factorial can become so large, it exceeds the maximum value of integer.

- **Modify the previous program and function to compute the factorial as floating-point number, while keeping the input to the function as an integer.**

Assessable questions

You may be called to present one of these questions during the last hour of the session, and this presentation will count for 5% of your final grade for the course. You will be called to do two such presentations during the trimester, at random times. Note that you will need to explain your solution to the demonstrator and make changes to the code if requested to demonstrate your understanding. Each question should take less than 5 minutes to complete, and you may be asked a few simple questions while presenting. If you are not present in the laboratory when you are due to present, you will receive 0 for this assessment item.

1. **Write a program that computes the volume of a sphere and prints the answer out. The user should enter the radius of the sphere. A function called *compute_sphere_volume* should be written that performs the computation and returns the value of the volume – this function should NOT perform the input/output. The prototype for this function is given below:**

   ```
   float compute_sphere_volume(float radius);
   ```

2. **Write a program to compute the reactance of a capacitor. The user should enter the capacitance C and frequency f. A function called compute_reactance should be written that performs the computation and returns the value of the reactance. The prototype for this function is given below:**

   ```
   double compute_cap_reactance(double cap, double freq);
   ```

   The main program should display the answer in scientific notation to 3 decimal places (Hint: The maths library <math.h> has a built-in constant for π called M_PI). The reactance of a capacitor is given by:

   $$X_c = \frac{1}{2\pi f C}$$

   Here is the required output (keyboard inputs are underlined):

```
This program computes the reactance of a given capacitor and
frequency.
Please enter the capacitance (in F): 10e-6
Please enter the frequency (in Hz): 1.5e3
The reactance of the capacitor is equal to 1.061e+01 ohms
```

3. **Write a C function which takes two integers as inputs, and returns the sum of all the numbers between those two integers (inclusive). For example if the inputs are 1 and 10, the output will be 55. If the first number is larger than the second, 0 should be returned instead.**

   Write a small main program which reads two numbers from the keyboard, passes these numbers to the function, then displays the result. Note that your function should *not* read anything from the keyboard, nor display anything to the screen, only the main function should do this.

4. **Write a C function which calculates integer powers of numbers. The function should take a float (the base x) and an integer (the index y) as inputs, and return the value x^y. For example, if the values 3.2 and 2 are entered, the output of the function should be 10.24. Your function should correctly return 1 if the index is 0, and the correct value if the index is negative (inverse of the positive index power). You should calculate the power using loops, not by calling any built-in C functions.**

   Test that the function works correctly by writing a small main function that reads values from the keyboard, passes them to the function, then displays the result. You should ensure that you test all cases (positive index, 0 index, negative index) to ensure they work correctly for various values.